

# 강의 4 실험

페이지 테이블 · MIT 6.1810 강의 4와 교재 3장을 xv6에서 직접 돌려 보며

MIT 6.1810(2026 가을)의 강의 4는 프로세스마다 따로 있는 주소 공간을 RISC-V의 페이지 테이블로 어떻게 만드는지 다룬다. 이 문서는 교재 3장의 내용을 xv6에서 직접 확인한 기록이다. gdb로 `satp` 레지스터와 물리 메모리를 읽어 주소 하나를 손으로 변환해 보고, QEMU 모니터로 커널과 쉘의 페이지 테이블 전체를 들여다봤다. 가드 페이지에 걸려 죽는 프로세스와 `sbrk`로 늘어나는 힙도 같은 방법으로 확인했다.

2026년 10월 7일 기록. 대상: xv6-riscv [06aad25](#)([lec1](#) 브랜치, 커널은 원본 그대로), 교재 2026-09-23판. 화면 그림은 모두 "tmux 캡처"다. QEMU와 gdb를 tmux 창에서 실제로 돌린 뒤 `tmux capture-pane`으로 받은 글자를 창 모양으로 그렸고, 글자는 고치지 않았다. 긴 출력은 줄 단위로 떨어 내고 그 자리에 ...를 넣었다. 밑줄 친 쪽 번호를 누르면 교재 PDF의 그 쪽이 열린다. 밑줄 친 소스 위치를 누르면 GitHub의 그 줄이 열린다(커밋 [06aad25](#)).

## 1. 범위: 강의 4는 교재의 어디인가

강의 4는 교재 3장 [Page tables](#) (31-42쪽)에 해당한다. 강의 노트의 순서는 하드웨어 → 커널 주소 공간 → 코드 읽기 → 사용자 주소 공간이고, 교재도 거의 같은 순서다.

| 강의 4 주제                                                          | 교재                        | 함께 볼 코드                                      |
|------------------------------------------------------------------|---------------------------|----------------------------------------------|
| 가상 주소, <code>satp</code> , Sv39, PTE                             | 3.1 (31-34쪽), 그림 3.1, 3.2 | <a href="#">kernel/riscv.h:246--417</a>      |
| 커널 주소 공간, 직접 매핑                                                  | 3.2 (34-36쪽), 그림 3.3      | <a href="#">kernel/memlayout.h</a>           |
| <code>kvmmake</code> , <code>mappages</code> , <code>walk</code> | 3.3 (36-37쪽)              | <a href="#">kernel/vm.c:22--175</a>          |
| 물리 페이지 할당                                                        | 3.4, 3.5 (37-38쪽)         | <a href="#">kernel/kalloc.c</a>              |
| 사용자 주소 공간, 가드 페이지                                                | 3.6 (38-39쪽), 그림 3.4      | <a href="#">kernel/proc.c:proc_pagetable</a> |
| <code>exec</code>                                                | 3.7 (39-41쪽)              | <a href="#">kernel/exec.c:kexec</a>          |
| 설계 이야기                                                           | 3.8 Real world (41쪽)      | -                                            |
| 연습 문제                                                            | 3.9 Exercises (42쪽)       | -                                            |

페이지 폴트로 프로세스가 죽는 장면(9절)은 4장 트랩과 5장 페이지 폴트로 이어진다. 그쪽 코드는 필요한 줄만 짰었다.

## 2. 실습 환경

강의 노트처럼 QEMU를 gdb 대기 상태로 띄우고, 다른 tmux 창에서 gdb를 붙였다. 출력을 단순하게 하려고 CPU는 하나만 썼다(-smp 1). 같은 창에서 `Ctrl-a c`를 누르면 xv6 콘솔과 QEMU 모니터를 오갈 수 있

다. 모니터의 `info mem`은 현재 `satp`가 가리키는 페이지 테이블의 매핑을 보여 준다.

```
# window 1
$ qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M \
  -smp 1 -nographic ... -S -gdb tcp::26200
# window 2
$ riscv64-unknown-elf-gdb kernel/kernel -ex 'target remote localhost:26200'
```

실험 전체는 같은 폴더의 `run-all.sh`에 순서대로 적어 두었고, 한 번 실행하면 `runs/`에 화면이 그대로 저장된다.

### 3. 페이지 테이블이 하는 일

CPU가 내보내는 주소는 모두 가상 주소다. MMU는 `satp` 레지스터가 가리키는 페이지 테이블을 보고 가상 주소를 물리 주소로 바꾼다. 페이지 테이블은 그냥 메모리에 있는 표이므로 커널이 값을 써서 고칠 수 있고, `satp`만 바꾸면 다른 주소 공간으로 갈아탈 수 있다.

RISC-V의 Sv39 방식은 64비트 주소 중 아래 39비트만 쓴다. 아래 12비트는 4KB 페이지 안의 위치이고, 그 위 27비트를 9비트씩 세 조각으로 나눠 세 단계 표를 차례로 찾는다.

|               | 63 ... 39                | 38 ... 30 | 29 ... 21 | 20 ... 12 | 11 ... 0      |
|---------------|--------------------------|-----------|-----------|-----------|---------------|
| 가상 주소 (64비트)  | EXT (25비트)<br>모두 0이어야 한다 | L2 (9비트)  | L1 (9비트)  | L0 (9비트)  | offset (12비트) |
| 0x10000000    | 0                        | 0x000     | 0x080     | 0x000     | 0x000         |
| 0x3fffffff000 | 0                        | 0x0ff     | 0x1ff     | 0x1ff     | 0x000         |

9비트 = 512칸 × 8바이트 = 4096바이트. 페이지 테이블 한 장이 정확히 한 페이지다.

그림 1. Sv39 가상 주소의 구성. 두 번째, 세 번째 줄은 이 문서에서 실제로 따라간 주소 두 개를 나눈 값이다. `PX(level, va)(kernel/riscv.h:411)`가 이 9비트 조각을 꺼낸다.

각 단계의 표는 512칸짜리 PTE 배열이고, PTE 하나는 8바이트다. 그래서 표 한 장이 정확히 한 페이지 (4096바이트)에 들어간다. 27비트를 한 번에 쓰는 평평한 표라면 프로세스마다  $2^{27} \times 8$ 바이트, 약 1GB가 필요하다. 세 단계로 나누면 실제로 쓰는 부분의 표만 만들면 된다.

핵심 페이지 테이블은 메모리에 있는 512칸짜리 표 세 단계다. `satp`가 맨 위 표를 가리키고, 가상 주소의 9비트 조각 세 개가 각 단계의 칸 번호다.

### 4. 실험 1: 페이징을 켜는 순간

xv6는 부팅 직후 페이징이 꺼진 상태로 `0x80000000`에서 시작한다. `main.c`가 `kvminit()`로 커널 페이지 테이블을 만든 뒤 `kvminithart()`에서 `satp`에 그 주소를 넣는 순간 페이징이 켜진다 (`kernel/vm.c:75--84`).

```
void
kvminithart()
{
    sfence_vma(); // wait for page-table writes
    w_satp(MAKE_SATP(kernel_pagetable)); // turn on paging
    sfence_vma(); // flush stale TLB entries
}
```

`kvminithart`에 중단점을 걸고 `satp`를 바꾸기 전후의 값을 읽었다.

```
riscv64-unknown-elf-gdb · kernel/kernel

(gdb) p/x $satp
$1 = 0x0
(gdb) next
78      sfence_vma();
(gdb) next
kvmminithart () at kernel/riscv.h:255
255      asm volatile("csrw satp, %0" : : "r"(x));
(gdb) next
kvmminithart () at kernel/vm.c:83
83      sfence_vma();
(gdb) p/x $satp
$2 = 0x80000000000087fff
(gdb) p/x (unsigned long)$satp >> 60
$3 = 0x8
(gdb) p/x ((unsigned long)$satp & 0xffffffff) << 12
$4 = 0x87fff000
(gdb) p/x kernel_pagetable
$5 = 0x87fff000
(gdb)
```

tmux 캡처. `w_satp` 전에는 0(페이징 꺼짐)이고, 뒤에는 `0x80000000000087fff`다.

`MAKE_SATP(kernel/riscv.h:248)`는 맨 위 4비트에 모드 8(Sv39)을, 아래에 페이지 테이블의 물리 페이지 번호를 넣는다. 그래서 `0x87fff`에 12비트를 붙이면 `kernel_pagetable`의 값 `0x87fff000`과 같다. `satp`에는 가상 주소가 아니라 물리 주소가 들어간다. MMU가 표를 찾아가는 데 다시 변환이 필요하면 끝이 나지 않기 때문이다.

**핵심** `satp` = 모드(Sv39는 8) + 맨 위 페이지 테이블의 물리 페이지 번호. 이 값을 쓰는 순간부터 CPU의 모든 주소가 변환된다.

## 5. 실험 2: 주소 하나를 손으로 변환하기

강의 노트를 따라 UART 레지스터 주소 `0x10000000`을 골랐다. 페이징이 켜진 직후에 gdb로 9비트 조각을 계산하고, 세 단계 표를 메모리에서 직접 읽었다. 커널은 물리 메모리를 같은 주소로 매핑해 두었으므로(6절) gdb에서 물리 주소를 그대로 읽을 수 있다.

```

riscv64-unknown-elf-gdb · kernel/kernel

(gdb) p/x (0x10000000 >> 30) & 0x1ff
$6 = 0x0
(gdb) p/x (0x10000000 >> 21) & 0x1ff
$7 = 0x80
(gdb) p/x (0x10000000 >> 12) & 0x1ff
$8 = 0x0
(gdb) x/gx 0x87fff000 + 0*8
0x87fff000: 0x0000000021fff801
(gdb) set $pte = *(unsigned long *) (0x87fff000 + 0*8)
(gdb) set $l1 = ($pte >> 10) << 12
(gdb) p/x $l1
$9 = 0x87ffe000
(gdb) x/gx $l1 + 0x80*8
0x87ffe400: 0x0000000021fff401
(gdb) set $pte = *(unsigned long *) ($l1 + 0x80*8)
(gdb) set $l0 = ($pte >> 10) << 12
(gdb) p/x $l0
$10 = 0x87ffd000
(gdb) x/gx $l0 + 0*8
0x87ffd000: 0x0000000040000007
(gdb) set $pte = *(unsigned long *) ($l0 + 0*8)
(gdb) p/x ($pte >> 10) << 12
$11 = 0x10000000
(gdb) p/x $pte & 0x3ff
$12 = 0x7
(gdb)

```

tmux 캡처. L2 → L1 → L0 순서로 PTE를 읽고, 마지막 PTE에서 물리 주소 0x10000000과 플래그 0x7을 얻었다.

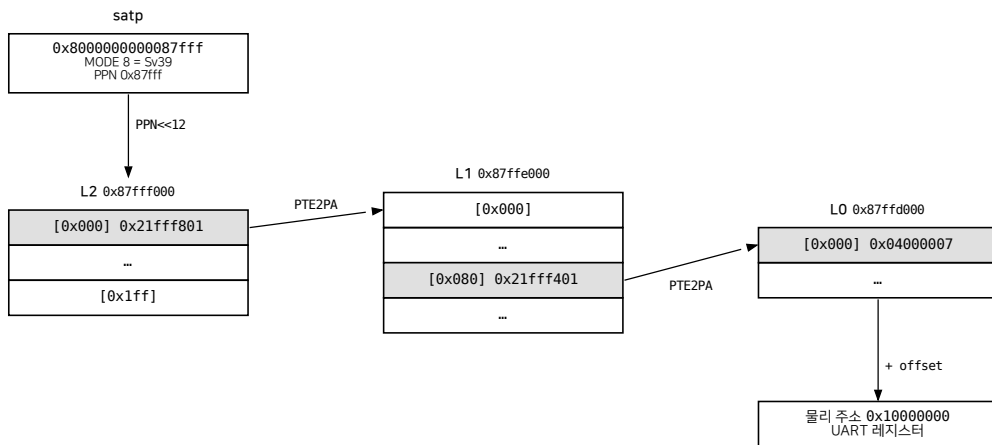


그림 2. 위 화면의 값을 그대로 옮긴 변환 경로. 회색 칸이 실제로 읽은 PTE다. 각 PTE의 PPN에 12비트를 붙이면 다음 단계 표의 물리 주소가 된다.

|            | 63..54 | 53 .. 10                    | 9   | 8 | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|------------|--------|-----------------------------|-----|---|---|---|---|---|---|---|---|---|
| PTE (64비트) | 예약     | PPN (44비트)<br>물리 페이지 번호     | RSW | D | A | G | U | X | W | R | V |   |
| 0x21fff801 | 0      | 0x87ffe → 다음 페이지 0x87ffe000 | 0   | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |
| 0x04000007 | 0      | 0x10000 → 물리 0x10000000     | 0   | 0 | 0 | 0 | 0 | 0 | 1 | 1 | 1 | 1 |

R, W, X가 모두 0이면 다음 단계 페이지 테이블을 가리키는 PTE이고, 하나라도 1이면 실제 페이지를 가리키는 마지막 PTE다.

그림 3. PTE의 비트 배치(kernel/riscv.h:395--406)와 위에서 읽은 PTE 두 개의 해석.

L2와 L1에서 읽은 PTE는 플래그가 V만 켜져 있다. R, W, X가 모두 0이면 “다음 표를 가리키는 PTE”라는 뜻이다. L0의 PTE 0x04000007은 V|R|W이고, PPN 0x10000에 12비트를 붙인 0x10000000이 최종 물리 주소다. 커널이 UART를 같은 주소에 매핑했으니(vm.c:30) 가상 주소와 물리 주소가 같게 나온 것이다.

하드웨어가 하는 이 과정을 소프트웨어로 똑같이 하는 함수가 `walk(kernel/vm.c:99)`다.

```
for (int level = 2; level > 0; level--) {
    pte_t *pte = &pagetable[PX(level, va)];
    if (*pte & PTE_V) {
        pagetable = (pagetable_t)PTE2PA(*pte);    // go down one level
    } else {
        if (!alloc || (pagetable = (pde_t *)kalloc()) == 0)
            return 0;
        memset(pagetable, 0, PGSIZE);
        *pte = PA2PTE(pagetable) | PTE_V;        // new page-table page
    }
}
return &pagetable[PX(0, va)];
```

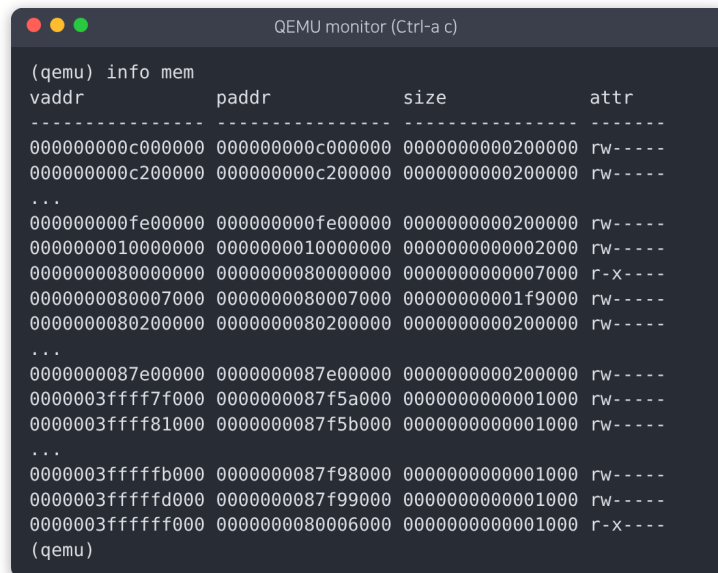
`PTE2PA(*pte)`로 얻은 물리 주소를 곧바로 포인터로 써서 다음 표를 읽는다. 이것도 직접 매핑 덕분이다. `walk`는 PTE의 주소를 돌려준다. `mappages(vm.c:147)`는 그 자리에 `PA2PTE(pa) | perm | PTE_V`를 써서 매핑을 만든다(`vm.c:168`).

핵심 가상 주소는 9비트씩 끊어 세 번 표를 찾아 바뀐다. 각 PTE의 PPN이 다음 표(마지막에는 실제 페이지)의 물리 주소다.

## 6. 커널 주소 공간

### 6.1 kvmmake가 만드는 것

커널 페이지 테이블은 `kvmmake(kernel/vm.c:22--53)`가 `kvmmap`을 여섯 번 부르고 `proc_mapstacks`로 커널 스택을 더해 만든다. 실험 1 직후 QEMU 모니터로 그 결과 전체를 봤다.



QEMU monitor (Ctrl-a c)

```
(qemu) info mem
```

| vaddr             | paddr            | size             | attr    |
|-------------------|------------------|------------------|---------|
| 00000000c0000000  | 00000000c0000000 | 0000000002000000 | rw----  |
| 00000000c2000000  | 00000000c2000000 | 0000000002000000 | rw----  |
| ...               |                  |                  |         |
| 00000000fe000000  | 00000000fe000000 | 0000000002000000 | rw----  |
| 0000000010000000  | 0000000010000000 | 0000000000002000 | rw----  |
| 0000000080000000  | 0000000080000000 | 0000000000007000 | r-x---- |
| 0000000080007000  | 0000000080007000 | 00000000001f9000 | rw----  |
| 0000000080200000  | 0000000080200000 | 0000000002000000 | rw----  |
| ...               |                  |                  |         |
| 0000000087e00000  | 0000000087e00000 | 0000000002000000 | rw----  |
| 00000003ffff7f000 | 0000000087f5a000 | 0000000000001000 | rw----  |
| 00000003ffff81000 | 0000000087f5b000 | 0000000000001000 | rw----  |
| ...               |                  |                  |         |
| 00000003fffffb000 | 0000000087f98000 | 0000000000001000 | rw----  |
| 00000003fffffd000 | 0000000087f99000 | 0000000000001000 | rw----  |
| 00000003fffff000  | 0000000080006000 | 0000000000001000 | r-x---- |

(qemu)

tmux 캡처. 커널 페이지 테이블의 `info mem`. 전체 163줄 중 일부만 남겼다. 원본은 `runs/infomem-kernel.txt`.

| 가상 주소                                 | 내용                       | 권한  | 물리 주소                  | 만드는 곳     |
|---------------------------------------|--------------------------|-----|------------------------|-----------|
| 0x3fffffff000 TRAMPOLINE              | 트램폴린 코드                  | r-x | 0x80006000 (커널 text 안) | vm.c:47   |
| 0x3fffffff000 KSTACK(0)               | proc[0]의 커널 스택           | rw- | kalloc 페이지             | proc.c:42 |
| 0x3ffffffc000                         | 가드 페이지 (매핑 없음)           | —   | —                      |           |
| ...                                   | KSTACK(1) ... KSTACK(63) | rw- | kalloc 페이지             |           |
| (비어 있음)                               |                          |     |                        |           |
| 0x80007000 etext ~ 0x88000000 PHYSTOP | 커널 데이터 + 남은 RAM          | rw- | 같은 주소<br>(직접 매핑)       | vm.c:42   |
| 0x80000000 KERNBASE ~ etext           | 커널 text                  | r-x |                        | vm.c:39   |
| 0x10001000 VIRTIO0                    | 디스크 장치 레지스터              | rw- |                        | vm.c:33   |
| 0x10000000 UART0                      | 콘솔 장치 레지스터               | rw- |                        | vm.c:30   |
| 0x0c000000 PLIC                       | 인터럽트 컨트롤러 (64MB)         | rw- |                        | vm.c:36   |
| 0x0                                   | 매핑 없음 (0번지 접근은 페이지 폴트)   |     |                        |           |

그림 4. 커널 주소 공간. 위 `info mem`과 `kernel/memlayout.h`, `kernel/vm.c`를 대조해 정리했다. 위쪽이 높은 주소다.

화면에서 확인한 것은 다음과 같다.

- **직접 매핑.** 장치와 RAM은 가상 주소와 물리 주소(vaddr, paddr)가 같다. 커널은 물리 주소를 포인터로 바꿔 바로 읽고 쓸 수 있다.
- **권한이 다르다.** 커널 코드 0x80000000~0x80007000은 r-x, 그 위 데이터와 남은 RAM은 rw-다. 코드는 고칠 수 없고 데이터는 실행할 수 없다. 경계 0x80007000은 링커 스크립트가 정한 etext다 (`kernel/kernel.ld`, nm으로 확인).
- **트램폴린은 두 번 매핑된다.** 맨 위 0x3fffffff000이 물리 0x80006000을 가리키는데, 이 물리 페이지는 직접 매핑된 커널 코드 안에도 있다.
- **커널 스택 사이에 빈칸이 있다.** 스택은 0x3ffffffd000, 0x3ffffffb000, ... 처럼 두 페이지 간격이다. KSTACK(p)(`memlayout.h:52`)가 사이에 매핑하지 않은 가드 페이지를 남기기 때문이다. 스택이 넘치면 페이지 폴트가 나고 커널이 멈춘다. 다른 프로세스의 스택을 조용히 덮어쓰지 않는다. 64줄은 NPR0C만 큼이다.
- 맨 아래 0번지 근처와 CLINT(0x02000000)는 매핑이 없다.

xv6는 4KB 페이지만 쓴다(mappages는 항상 LO PTE를 만든다). `info mem`은 주소와 권한이 이어지는 페이지를 한 줄로 묶어 보여 주는데, 이번 출력에서는 2MB 경계마다 줄이 나뉘었다.

**핵심** 커널은 RAM과 장치를 같은 주소로 매핑(직접 매핑)해 물리 주소를 포인터로 쓴다. 예외는 맨 위의 트램폴린과 커널 스택이고, 스택 사이에는 가드 페이지가 있다.

## 7. 물리 페이지 할당기

실험 2의 세 표는 0x87fff000, 0x87ffe000, 0x87ffd000으로 RAM의 맨 위부터 한 페이지씩 내려왔다. 이유는 `kernel/kalloc.c`에 있다. 빈 페이지는 각 페이지의 첫 8바이트를 next 포인터로 쓰는 연결 리스트로 관리한다.

```
void kinit() { ... freerange(end, (void *)PHYSTOP); } // kalloc.c:27
// freerange: kfree() every page from end up to PHYSTOP, in ascending order
```

```
kfree(pa): memset(pa, 1, PGSIZE); r->next = kmem.freelist; kmem.freelist = r;
kalloc(): r = kmem.freelist; kmem.freelist = r->next; memset(r, 5, PGSIZE);
```

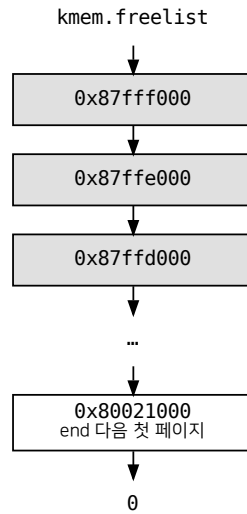


그림 5. kinit 직후의 빈 페이지 목록. 낮은 주소부터 차례로 kfree해 앞에 끼워 넣으므로 가장 높은 페이지가 맨 앞에 온다.

kvmmake가 처음 받은 세 페이지(회색)가 실험 2의 L2, L1, L0 표다.

end는 커널 이미지 바로 뒤 주소(0x80020bb0, nm으로 확인)이고, 올림하면 첫 빈 페이지 0x80021000이 된다. kfree는 페이지를 1로, kalloc은 5로 채운다. 해제한 페이지를 실수로 계속 쓰는 코드가 엉뚱한 값을 읽어 빨리 드러나게 하려는 것이다.

핵심 kalloc은 4KB 페이지를 연결 리스트의 맨 앞에서 꺼내 준다. 페이지 테이블, 커널 스택, 사용자 메모리가 모두 여기서 나온다.

## 8. 사용자 주소 공간

### 8.1 쉘의 페이지 테이블 찾기

셸까지 부팅한 뒤 gdb로 proc[]를 읽었다. sh는 pid 2이고 페이지 테이블은 0x87f41000, 크기 p->sz는 0x5000이다.

```

riscv64-unknown-elf-gdb · kernel/kernel
(gdb) p proc[1].name
$13 = "sh", '\000' <repeats 13 times>
(gdb) p proc[1].pid
$14 = 2
(gdb) p proc[1].state
$15 = SLEEPING
(gdb) p/x proc[1].pagetable
$16 = 0x87f41000
(gdb) p/x proc[1].sz
$17 = 0x5000
(gdb)
  
```

tmux 캡처. 입력을 기다리며 잠든(SLEEPING) sh의 struct proc.

이 표를 QEMU 모니터로 보려면 CPU의 satp가 sh의 것이어야 한다. 커널은 사용자 모드로 돌아가기 직전, 트램펄린의 userret에서 satp를 바꾼다(kernel/trampoline.S:110--112). 그 바로 다음 명령의 가상

주소 0x3fffffff0ac(TRAMPOLINE + userret의 오프셋 0x9c + 명령 4개)에 하드웨어 중단점을 걸고, xv6 콘솔에 echo hi를 쳐서 sh를 깨웠다.

```
riscv64-unknown-elf-gdb · kernel/kernel

(gdb) break prepare_return
Breakpoint 2 at 0x80002448: file kernel/trap.c, line 102.
(gdb) continue
Continuing.

Breakpoint 2, prepare_return () at kernel/trap.c:102
102      {
(gdb) p cpus[0].proc->name
$18 = "sh", '\000' <repeats 13 times>
(gdb) p cpus[0].proc->pid
$19 = 2
(gdb) p/x cpus[0].proc->pagetable
$20 = 0x87f41000
(gdb) hbreak *0x3fffffff0ac
Hardware assisted breakpoint 3 at 0x3fffffff0ac
(gdb) continue
Continuing.

Breakpoint 3, 0x0000003fffffff0ac in ?? ()
(gdb) p/x $satp
$21 = 0x800000000087f41
(gdb)
```

tmux 캡처. prepare\_return에서 pid 2(sh)를 확인하고, 트램펄린에서 멈췄을 때 satp가 0x87f41을 가리킨다.

```
QEMU monitor (Ctrl-a c)

(qemu) info mem
vaddr      paddr      size      attr
-----
0000000000000000 0000000087f3e000 0000000000001000 r-xu-a-
00000000000001000 0000000087f3b000 0000000000001000 r-xu---
00000000000002000 0000000087f3a000 0000000000001000 rw-u-ad
00000000000003000 0000000087f39000 0000000000001000 rw-----
00000000000004000 0000000087f38000 0000000000001000 rw-u-ad
0000003fffffe000 0000000087f55000 0000000000001000 rw---ad
0000003fffffff000 0000000080006000 0000000000001000 r-x--a-
(qemu)
```

tmux 캡처. 이때의 info mem, 곧 sh의 사용자 페이지 테이블 전체다.

| 가상 주소                   | 내용                   | info mem 속성 | 만드는 곳        |
|-------------------------|----------------------|-------------|--------------|
| 0x3fffffff000           | 트램펄린 (커널과 같은 물리 페이지) | r-x--a-     | proc.c:189   |
| 0x3fffffe000            | 트랩프레임                | rw---ad     | proc.c:197   |
| (비어 있음: sbrk로 힘이 자랄 자리) |                      |             |              |
| 0x4000                  | 사용자 스택 (1페이지)        | rw-u-ad     | exec.c:91-95 |
| 0x3000                  | 가드 페이지 (U 비트 없음)     | rw----      |              |
| 0x2000                  | 데이터, bss             | rw-u-ad     | exec.c:72-76 |
| 0x0000, 0x1000          | text (코드)            | r-xu-a-     |              |

속성 글자 순서: r w x u g a d. u가 없으면 사용자 모드에서 접근할 수 없다. p->sz = 0x5000.

그림 6. sh의 사용자 주소 공간. 위 화면의 일곱 줄을 그대로 정리했다. 위쪽이 높은 주소다.

- 0번지부터 코드, 데이터가 오고, 그 위에 가드 페이지와 스택이 한 장씩 있다. 커널 페이지 테이블과 달리 UART나 커널 코드는 없다. 사용자 프로그램은 자기 페이지 외에는 주소조차 가질 수 없다.
- 사용자 페이지에는 u가 붙고, 트램펄린과 트랩프레임에는 없다. 둘은 같은 주소 공간에 있지만 커널 모드에서만 쓴다. 트램펄린의 물리 주소 0x80006000은 커널 쪽과 같은 페이지다.
- a(accessed), d(dirty)는 xv6가 아니라 하드웨어(여기서는 QEMU)가 켜다. 두 번째 코드 페이지는 아직 실행되지 않아 a가 없다.

같은 가상 주소 0번지라도 프로세스마다 다른 물리 페이지를 가리킨다. 10절에서 본 sh의 자식(pid 3)은 0번지가 0x87f47000에 매핑되어 있었다.

핵심 프로세스마다 페이지 테이블이 따로 있고, 커널은 사용자로 돌아갈 때 satp를 그 표로 바꾼다. U 비트가 없는 페이지는 같은 주소 공간에 있어도 사용자 코드가 건드릴 수 없다.

## 9. exec와 가드 페이지

### 9.1 exec가 주소 공간을 만드는 순서

그림 6의 배치는 kexec(kernel/exec.c:28)가 만든다. ELF 파일의 LOAD 세그먼트를 읽어 그 가상 주소에 페이지를 할당하고 내용을 채운다. sh의 세그먼트를 호스트에서 확인하면 그림 6의 코드(0x0-0x13e9, 두 페이지)와 데이터(0x2000) 칸과 정확히 맞는다.

```
$ riscv64-unknown-elf-readelf -lW user/_sh
Type   Offset   VirtAddr           ... FileSiz  MemSiz  Flg Align
LOAD   0x001000 0x0000000000000000 ... 0x0013e9 0x0013e9 R E 0x1000
LOAD   0x003000 0x0000000000002000 ... 0x000010 0x000098 RW 0x1000
```

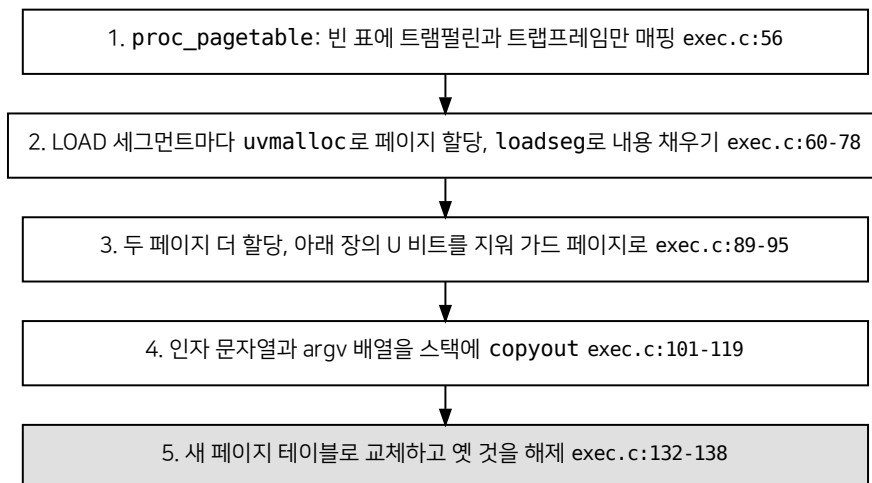


그림 7. kexec의 순서. 새 페이지 테이블을 끝까지 만든 다음에야 옛 것과 바꾸므로, 중간에 실패해도 원래 프로그램은 그대로 남는다.

loadseg(exec.c:157)는 walkaddr로 사용자 가상 주소에 해당하는 물리 주소를 얻은 뒤, 그 물리 주소에 파일 내용을 바로 읽어 넣는다. 데이터 위에는 두 페이지를 더 할당하고(exec.c:91), 아래쪽 한 장에서 uvmclear(vm.c:331)로 PTE\_U만 지운다. 그래서 가드 페이지는 V는 켜져 있지만 사용자 모드에서는 접근할 수 없다. 그림 6에서 0x3000이 rw-----로 보인 이유다.

## 9.2 실험 3: 스택 아래를 읽으면

usertests의 stacktest는 자식 프로세스에서 스택 포인터보다 한 페이지 아래를 읽는다. 별도의 QEMU를 띄워 실행했다.

```
xv6 · qemu-system-riscv64

$ usertests stacktest
usertests starting
test stacktest: usertrap(): unexpected scause 0xd pid=5
                sepc=0x1c44 stval=0x10e90
OK
ALL TESTS PASSED
$
```

tmux 캡처. 가드 페이지를 읽은 자식 프로세스가 커널에 의해 죽고, 테스트는 그것을 확인하고 통과한다.

- scause 0xd(13)는 읽기 페이지 폴트다. stval은 문제의 가상 주소 0x10e90이다.
- usertests는 데이터가 0xfd08에서 끝나므로(readelf: 0x9000 + 0x6d08) 가드 페이지가 0x10000, 스택이 0x11000이다. 0x10e90은 가드 페이지 안이다.
- usertrap(kernel/trap.c:72--78)은 페이지 폴트를 먼저 vmfault에 넘긴다. vmfault는 이미 매핑된 페이지면 0을 돌려주고(vm.c:466), 가드 페이지는 V가 켜져 있으므로 거절된다. 그래서 메시지를 찍고 프로세스를 죽인다.

핵심 가드 페이지는 스택 바로 아래의 u 비트 없는 페이지다. 스택이 넘쳐 그곳을 건드리면 다른 데이터를 덮는 대신 페이지 폴트가 나고 프로세스가 죽는다.

## 10. 실험 4: sbrk로 힙이 자라는 모습

셸은 명령을 받으면 fork한 자식에서 명령을 파싱한다(user/sh.c:172--173). 파싱에 쓰는 malloc이 처음 불리면 sbrk로 메모리를 늘린다. sys\_sbrk에 중단점을 걸고 echo hi를 처리하는 과정을 봤다.

```
riscv64-unknown-elf-gdb · kernel/kernel

(gdb) delete 2 3
(gdb) break sys_sbrk
Breakpoint 4 at 0x80002936: file kernel/sysproc.c, line 41.
(gdb) continue
Continuing.

Breakpoint 4, sys_sbrk () at kernel/sysproc.c:41
41      {
(gdb) p cpus[0].proc->name
$22 = "sh", '\000' <repeats 13 times>
(gdb) p cpus[0].proc->pid
$23 = 3
(gdb) p/x cpus[0].proc->sz
$24 = 0x5000
(gdb) p/x cpus[0].proc->trapframe->a0
$25 = 0x10000
(gdb) finish
Run till exit from #0  sys_sbrk () at kernel/sysproc.c:41
0x000000008000289c in syscall () at kernel/syscall.c:146
146      p->trapframe->a0 = syscalls[num]();
Value returned is $26 = 20480
(gdb) p/x cpus[0].proc->sz
$27 = 0x15000
(gdb)
```

tmux 캡처. sh의 자식(pid 3)이 0x10000바이트를 요청했다. 이전 크기 20480(0x5000)을 돌려주고, p->sz는 0x15000이 됐다.

사용자 라이브러리의 `sbrk()`(`user/ulib.c:153`)는 `SBRK_EAGER`로 시스템 콜을 부른다. 그래서 커널의 `sys_sbrk(kernel/sysproc.c:40)`는 `growproc` → `uvmalloc`으로 바로 페이지를 할당하고 PTE를 만든다. 이 자식이 사용자 모드로 돌아가는 순간 다시 `info mem`을 봤다.

```

(qemu) info mem
vaddr          paddr          size          attr
-----
0000000000000000 0000000087f47000 0000000000001000 r-xu-a-
00000000000001000 0000000087f44000 0000000000001000 r-xu-a-
00000000000002000 0000000087f45000 0000000000001000 rw-u-ad
00000000000003000 0000000087f48000 0000000000001000 rw----
00000000000004000 0000000087f37000 0000000000001000 rw-u-ad
00000000000005000 0000000087f36000 0000000000001000 rw-u--
00000000000006000 0000000087f35000 0000000000001000 rw-u--
00000000000007000 0000000087f34000 0000000000001000 rw-u--
00000000000008000 0000000087f33000 0000000000001000 rw-u--
00000000000009000 0000000087f32000 0000000000001000 rw-u--
0000000000000a000 0000000087f31000 0000000000001000 rw-u--
0000000000000b000 0000000087f30000 0000000000001000 rw-u--
0000000000000c000 0000000087f2f000 0000000000001000 rw-u--
0000000000000d000 0000000087f2e000 0000000000001000 rw-u--
0000000000000e000 0000000087f2d000 0000000000001000 rw-u--
0000000000000f000 0000000087f2c000 0000000000001000 rw-u--
00000000000010000 0000000087f2b000 0000000000001000 rw-u--
00000000000011000 0000000087f2a000 0000000000001000 rw-u--
00000000000012000 0000000087f29000 0000000000001000 rw-u--
00000000000013000 0000000087f28000 0000000000001000 rw-u--
00000000000014000 0000000087f27000 0000000000001000 rw-u--
00000003ffffffe000 0000000087f42000 0000000000001000 rw---ad
00000003ffffff000 0000000080006000 0000000000001000 r-x--a-
(qemu)

```

tmux 캡처. 스택 위 `0x5000--0x14fff`에 새 페이지 16장이 생겼다. 아직 쓰지 않아 `a`, `d`가 없다.

새 페이지의 물리 주소는 `0x87f36000`, `0x87f35000`, ... 처럼 한 장씩 내려간다. 7절의 빈 페이지 목록에서 앞에서부터 꺼냈기 때문이다. 가상 주소는 이어져 있어도 물리 주소가 이어질 필요는 없다. 지금의 `xv6`에는 `sbrklazy()`도 있어서, 이때는 크기만 늘리고 페이지는 처음 접근할 때 `vmfault`가 만든다(`vm.c:459`).

핵심 `sbrk`는 `p->sz`를 늘리고 그 범위에 새 페이지와 `U` 비트가 있는 PTE를 만든다. 가상 주소는 이어지지 만 물리 페이지는 어디서든 온다.

## 11. 정리

1. 페이지 테이블은 4KB 페이지 단위로 가상 주소를 물리 주소로 바꾸는 표이고, `satp`가 맨 위 표의 물리 주소를 가리킨다.
2. `Sv39`은 가상 주소 39비트를 9비트씩 세 번 나눠 세 단계 표를 찾는다. 표 한 장은 512칸, 정확히 한 페이지이다.
3. PTE는 다음 표나 실제 페이지의 물리 페이지 번호와 `V`, `R`, `W`, `X`, `U` 같은 권한 비트로 이뤄진다.
4. 커널은 RAM과 장치를 직접 매핑해 물리 주소를 포인터로 쓴다. 트램펄린과 커널 스택만 높은 주소에 따로 매핑한다.
5. 물리 페이지는 `kalloc`의 연결 리스트에서 한 장씩 나온다.
6. 프로세스마다 페이지 테이블이 따로 있고, `U` 비트로 사용자 모드에서 쓸 수 있는 페이지를 구분한다.
7. `exec`는 새 주소 공간을 다 만든 뒤 바꿔 끼우고, 스택 아래에는 가드 페이지를 둔다. `sbrk`는 그 위로 힙을 늘린다.